

Students with 30% reduction can omit: Point 2 in Exercise B (Ranking), Scenario 2 in Exercise C (Physical databases) and Point c of Exercise D (Triggers).

A. Concurrency Control (9 points)

Present a schedule S consisting of as few write operations as possible and no read operation such that the schedule is a member of 2PL and TS' (i.e., TS with Thomas Rule), but not a member of Strict 2PL, TS Mono and TS Multi.

1. Justify memberships and non-memberships of S .
2. Argue why no smaller schedule would have the same (non-)memberships.

NB For TS-Multi, adopt the convention according to which the request $w_{ts}(x)$ is rejected if $ts < \text{WTM}_N(x)$.

Solution.

1. Several choices are possible. A schedule satisfying all requirements is the following:

$w_2(x) \quad w_1(x) \quad w_2(y)$

2PL

	1	2	3
x	$\uparrow_2 w_2$	$\downarrow_2$	$\uparrow_1 w_1 \downarrow_1$
y	$\uparrow_2$		$w_2 \downarrow_2$
	2	1	

$\uparrow$: write lock request
 $\downarrow$: unlock request

Strict 2PL

	1	2	3
x	w_2	$\downarrow_2$	w_1
y			$w_2 \downarrow_2^2$

$\downarrow$: mandatory unlock request
 $\downarrow$: commit time

TS Mono

Operation	RTM(x)	WTM(x)	RTM(y)	WTM(y)	Killed transactions
$w_2(x)$	0 (0)	2	0 (0)	0	
$w_1(x)$					1 as $1 < \text{WTM}(x) = 2$
$w_2(y)$				2	

TS' (TS + Thomas Rule)

Operation	RTM(x)	WTM(x)	RTM(y)	WTM(y)	Killed transactions
$w_2(x)$	0 (0)	2	0 (0)	0	
$w_1(x)$					skipped
$w_2(y)$				2	

TS Multi (killing out-of-time writes)

Operation	RTM(x)	WTM(x)	RTM(y)	WTM(y)	Killed transactions
$w_2(x)$	0 (0)	0, 2	0 (0)	0	
$w_1(x)$					1 as $1 < \text{last WTM}(x) = 2$
$w_2(y)$				0, 2	

- The reason why no schedule with less than three operations could satisfy all the constraints is as follows. In order to have a conflict at least two transactions are needed, with at least one operation each. For the conditions of Strict 2PL to be violated, at least one transaction needs to have at least two operations, since there must be a lock release (following some operation) placed before commit time, which means that a later operation by the same transaction must exist in the schedule. Therefore, at least three operations are needed (at least two transactions, one of which with at least two operations).

B. Ranking (6 points)

A group of journalists is trying to determine the best male tennis player of the Open Era (i.e., since 1968). They base their analysis on the following tables, reporting wins in the four Grand Slam tournaments: Australian Open (AO), Roland-Garros French Open (RG), US Open (US), and Wimbledon (WI). Clearly, more wins are better than fewer wins.

Table 1 for Point 1.

	<i>AO</i>	<i>RG</i>	<i>US</i>	<i>WI</i>
Agassi:	4	1	2	1
Becker:	2	0	1	3
Borg:	0	6	0	5
Connors:	1	0	5	0
Djokovic:	10	2	3	7
Edberg:	2	0	2	2
Federer:	6	1	5	8
Lendl:	2	3	3	0
McEnroe:	0	0	4	3
Nadal:	2	14	4	2
Sampras:	2	0	5	7
Wilander:	3	3	1	0

Table 2 for Point 2.

<i>AO</i>	<i>RG</i>	<i>US</i>	<i>WI</i>
Djokovic: 10	Nadal: 14	Sampras: 5	Federer: 8
Federer: 6	Borg: 6	Federer: 5	Sampras: 7
Agassi: 4	Wilander: 3	Connors: 5	Djokovic: 7
Wilander: 3	Lendl: 3	Nadal: 4	Borg: 5
Sampras: 2	Djokovic: 2	McEnroe: 4	McEnroe: 3
Nadal: 2	Federer: 1	Lendl: 3	Becker: 3
Lendl: 2	Agassi: 1	Djokovic: 3	Nadal: 2
Edberg: 2	Sampras: 0	Edberg: 2	Edberg: 2
Becker: 2	McEnroe: 0	Agassi: 2	Agassi: 1
Connors: 1	Edberg: 0	Wilander: 1	Wilander: 0
McEnroe: 0	Connors: 0	Becker: 1	Lendl: 0
Borg: 0	Becker: 0	Borg: 0	Connors: 0

1. In order to provide a good starting point for the journalists, please identify the skyline of the dataset. Just indicate the elements of the skyline; no intermediate steps need to be specified.
2. Given the content of the skyline, the journalists decide to place more weight on European competitions, and so they come up with the following scoring function: $s(o) = o.AO + 2 \cdot o.RG + o.US + 2 \cdot o.WI$. Apply the threshold algorithm to the table above (on the right-hand side) to identify the top-1 player according to the given scoring function. Specify, for each execution round of the algorithm, the threshold and the content of the buffer. No need to indicate all the accesses.

Solution.

1. The skyline contains 5 players: Djokovic, Federer, Wilander, Nadal, Borg. All the others are dominated by at least one other player: Agassi, Edberg and Becker are dominated by Djokovic; Lendl is dominated by Nadal; Sampras, Connors and McEnroe are dominated by Federer.
2. Nadal is the top-1 player according to the given scoring function.
 - Round 1: the buffer contains Nadal with a score of $2 + 2 \cdot 14 + 4 + 2 \cdot 2 = 38$, while the threshold is $10 + 2 \cdot 14 + 5 + 2 \cdot 8 = 59$.
 - Round 2: the buffer contains again Nadal, but the threshold is now $6 + 2 \cdot 6 + 5 + 2 \cdot 7 = 37 < 38$, so TA stops.

C. Physical Databases (9 points)

A retail company records data about customers and receipts of its grocery stores. Table `CUSTOMER` (`CUSTOMERID`, `LastName`, `Name`, `City`, ...) contains 30K tuples over 5K blocks, with a primary hash structure built on `CUSTOMERID` and filling factor below 50%. Table `RECEIPT` (`RECEIPTID`, `Timestamp`, `StoreID`, `CustomerID`, `TotalAmount`) records 10M tuples, primarily stored sequentially ordered by `CustomerID` on 100K blocks. We know that $\text{val}(\text{StoreID})=2\text{K}$ and $\text{val}(\text{City})=10$.

For the following query in the three scenarios below, describe briefly (but precisely) a reasonably efficient query plan and estimate its execution cost, possibly evaluating also alternative plans.

```
SELECT *
FROM CUSTOMER
WHERE City = 'Milan' AND CUSTOMERID IN
      (SELECT CustomerID
       FROM RECEIPT
       WHERE StoreID = 'MI-11' )
```

1. There are no secondary access structures;
2. Table `RECEIPT` has a secondary hash index built on `CustomerID`, with the same hash function used for the primary structure of `CUSTOMER`, with overflow cost 1.3 and 6.5K blocks (including the overflow blocks);
3. Table `RECEIPT` also has a B+ index built on `StoreID` as search key, with $F=90$ and 50K leaf nodes.

Solution.

1. SCENARIO A

- **Strategy 1 (scan RECEIPT, filter, and lookup)**

Scan `RECEIPT` and filter the tuples satisfying `StoreID='MI-11'`; for the `CustomerID`s of these tuples, access the `CUSTOMER` tuples using the hash function. Check condition `City='Milan'` and return all the attributes of the customers.

Cost 1: $100\text{K} (\text{scan RECEIPT}) + (10\text{M}/2\text{K}) (\text{filtered tuples}) * 1 (\text{lookup in CUSTOMER using h}) = 100\text{K} + 5\text{K} * 1 \approx \mathbf{105\text{K}}$ I/O accesses

- **Strategy 2 (scan CUSTOMER, filter, and lookup) - too expensive**

Scan `CUSTOMER`, filter the tuples satisfying `City='Milan'`; for each tuple, scan the primary sequential structure of `RECEIPT` to compute the nested query. The search of `CustomerID` in the sequential structure can stop as soon as all the tuples of the `CustomerID` have been found: if we consider the average case, we scan 50K blocks.

Cost 2: $5\text{K} (\text{scan CUSTOMER}) + (30\text{K}/10) (\text{filtered tuples}) * 50\text{K} (\text{lookup RECEIPT}) = 5\text{K} + 3\text{K} * 50\text{K} \approx \mathbf{150\text{M}}$ I/O accesses

$30\text{K}/10 = 3\text{K}$ tuples occupy $5\text{K}/10 = 500$ blocks (for the `select *` we need the full `CUSTOMER` tuples). This plan cannot be improved. Strategy 1 is better.

2. SCENARIO B

- **Strategy 3 (hash join)**

A hash join can be applied on the primary structure of `CUSTOMER` and the hash index of `RECEIPT`. The attributes of the customers are already available and can be filtered by `City='Milan'`: for the $30\text{K}/10$ customers from Milan, check the condition `StoreID='MI-11'` in the primary sequential structure of `RECEIPT`.

The hash index contains all the pointers to the full tuples: that plan needs to follow them to check the condition `StoreID='MI-11'`. How many blocks of `RECEIPT` are accessed? For each customer, there are on average 10M receipts/30K customers. For 3K customers, there will be 1M receipts, therefore we need to follow 1M pointers (and load 1M blocks) to get the full tuples.

Cost 3: $5K + 6.5K$ (hash join) + $1M$ (pointers to follow) $\approx$ **1M** I/O accesses

- **Strategy 4 (hash join + caching)**

Apply the hash join like in strategy 3 and filter the customers by `City='Milan'`: for these 3K customers, their `RECEIPTs` must be loaded to check the `StoreID`.

Notice that the tuples in the primary structure of `RECEIPT` are ordered by `CustomerID`: for each customer, the receipts (10M receipts/30K customers = 333 receipts/customer) are stored in contiguous blocks. In how many blocks? One block contains (10M tuples/ 100K blocks =) 100 tuples/block: 333 tuples are contained in 4 blocks. For each customer, we need to read 4 blocks: in the hash index, the 333 pointers to these blocks will point to the same 4 blocks; if the block is already in main memory, it is not loaded again.

Cost 4: $5K + 6.5K$ (hash join) + $3K$ (customers from Milan) * 4 (`RECEIPT` blocks for each customer) = **23,5K** I/O

- **Strategy 5 (scan CUSTOMER and lookup RECEIPT)**

Scan `CUSTOMER` and filter the customers by `City='Milan'`: for these 3K customers use the hash index to get the 333 pointers to `RECEIPTs` to check the `StoreID`.

Cost 5: $5K$ (scan `CUSTOMER`) + $3K$ (customers from Milan) * ($1,3$ (cost to access the hash index of `RECEIPT`) + 333 (follow the pointers to the receipts of the customer)) $\approx$ **1M** I/O

- **Strategy 6 (scan CUSTOMER and lookup RECEIPT + caching)**

Scan `CUSTOMER` and filter the customers by `City='Milan'`: for these 3K customers use the hash index to get the 333 pointers to `RECEIPTs` to check the `StoreID`. These 333 pointers will point to 4 blocks (see Strategy 4). If we exploit caching, the cost becomes:

Cost 6: $5K$ (scan `CUSTOMER`) + $3K$ (customers from Milan) * ($1,3$ (cost to access the hash index of `RECEIPT`) + 4 (`RECEIPT` blocks for each customer)) = **20,9K** I/O

3. SCENARIO C

- **Strategy 7 (Access StoreID='MI-11' tuples through the B+, then lookup customers)**

Search 'MI-11' using the B+ tree secondary index. To store 10M pointers, the index needs 4 levels. The 50K leaf nodes contain 10M pointers: each leaf node contains 10M pointers/50K leaf nodes = 200 pointers/leaf nodes.

Since $\text{val}(\text{StoreID})=2K$, for each `StoreID` there are on average 10M pointers/ 2K stores = 5K pointers/store.

For store 'MI-11', the leaf nodes contain 5K pointers, stored in (5K pointers/200 pointers/leaf node =) 25 leaf nodes (26, if we consider that the searched key does not start at the beginning of the block).

Cost 7: 3 (intermediate levels) + 25 (or 26 leaf nodes) + $5K$ (pointers to follow to get the `CustomerID`) + $5K$ (for each `CustomerID` found in the `RECEIPT` tuples) * 1 (cost to access the `CUSTOMER` tuple in the hash primary structure) = $3 + 25$ (or 26) + $5K + 5K*1 \approx$ **10K** I/O accesses

D. Trigger (8 points)

A mobile app statistics application manages data stored in a database with the schema reported below. Primary keys are represented with capital letters.

```
COMPANY (COMPANYID, name, address, mail)
APPS (APPID, companyId, tot_installs, tot_uninstalls)
APP_WEEKLY_STATS (APPID, WEEK_NUM, week_installs, week_uninstalls)
ALERT (APPID, WEEK_NUM, delta)
BEST_APPS (APPID, COMPANYID)
```

At the end of each week, an application inserts in table `APP_WEEKLY_STATS` the number of installations and uninstallations of all the apps of all companies occurred in the current week. Some triggers react to the insertion of the weekly statistics of the apps by: a) Updating the total (i.e., cumulative) number of installations and uninstallations of the apps. b) Creating an alert (by inserting a row in the `ALERT` table) if the uninstallations of an app have increased more than 10% with respect to the number of uninstallations observed in the preceding week. The alert reports the number of uninstallations exceeding the number of uninstallations observed the week before (in the `delta` column). c) Updating, only if needed, the `BEST_APP` table recording, for each company, the app with the highest number of total installations.

Hypotheses:

- Weeks of the year are integers from 1 to 52.
- The available apps are fixed and all stored in the `APPS` table. No new apps nor dismissed apps.
- Ignore the case in which two or more apps of the same company have exactly the same number of installations.

Specify any assumption on the database that you may have added to make your triggers work.

Solution.

```
create trigger INSERTWEEKLYSTATS
  after insert
  on APP_WEEKLY_STATS
  for each row
begin
  update APPS -- assumes tot_installs and tot_uninstalls initialized to 0
  SET tot_installs = tot_installs + NEW.week_installs,
      tot_uninstalls = tot_uninstalls + NEW.week_uninstalls
  WHERE APPID = NEW.APPID;
end;

create trigger CREATEALERT
  after insert
  on APP_WEEKLY_STATS
  for each row
  when (NEW.WEEK_NUM > 1) -- assumes comparison only within the year
DECLARE previous_uninstalls INTEGER;
begin
  select week_uninstalls into previous_uninstalls
  from APP_WEEKLY_STATS
  where APPID = NEW.APPID and WEEK_NUM = NEW.WEEK_NUM - 1;
  -- week_uninstalls increased more than 10% wrt previous week
  IF NEW.week_uninstalls > 1.10 * previous_uninstalls THEN
    INSERT INTO ALERT
      VALUES (NEW.APPID, NEW.WEEK_NUM, NEW.week_uninstalls - previous_uninstalls);
  END IF;
end;
```

```

create trigger UPDATEBESTAPP
  after update
  of tot_installs
  on APPS
  for each row
begin
  -- no best app is present
  IF 0 = (select count(*) from BEST_APPS where COMPANYID = NEW.companyId) THEN
    insert into BEST_APPS values (NEW.APPID, NEW.companyId);
  -- need to change the best app for this companyId
  ELSIF NEW.APPID <> (select APPID from BEST_APPS where COMPANYID = NEW.companyId) AND
    NEW.tot_installs > (select tot_installs
                        from APPS A JOIN BEST_APPS BA on APPID
                        where BA.COMPANYID = NEW.companyId) THEN
    update BEST_APPS set APPID = NEW.APPID where COMPANYID = NEW.companyId;
  END IF;
end;

```